

ShopVerse

Complete Stack Startup & Data Access Guide

Java 21 · Spring Boot 3 · PostgreSQL · Redis · MongoDB
Cassandra · Neo4j · Elasticsearch · Kafka · RabbitMQ
Prometheus · Grafana · Docker

Section 1 - Prerequisites

Ensure the following tools are installed before starting the ShopVerse stack:

Tool	Minimum Version	Download / Install
Java (Temurin)	21+	https://adoptium.net
Maven	3.9+	https://maven.apache.org
Node.js	18+	https://nodejs.org
Docker Desktop	4.x+	https://www.docker.com/products/docker-desktop
Git	2.x+	https://git-scm.com
redis-cli	Bundled	Included in Docker or: brew install redis
mongosh	2.x+	Bundled in Docker or https://www.mongodb.com/try/download/shell
cqlsh	Bundled	Included in Docker image cassandra:4.1
kafka-console-*	Bundled	Included in confluentinc/cp-kafka:7.6.0
curl	Any	Pre-installed on macOS/Linux; use Git Bash on Windows

Section 2 - Starting the Full Stack

All services are defined in **docker-compose.yml** at the project root **F:\Final\ShopVerse**. Docker Desktop must be running before executing any command.

Start all services at once:

```
cd F:\Final\ShopVerse
docker-compose up -d
```

Start only infrastructure (no app/UI):

```
docker-compose up -d postgres redis mongo cassandra elasticsearch zookeeper kafka rabbitmq neo4j
prometheus grafana nginx
```

Stop all services:

```
docker-compose down
```

Service Port Reference:

Service	Container Name	Port(s)	Health Check
PostgreSQL 16	shopverse-postgres	5432	pg_isready -U shopverse
Redis 7	shopverse-redis	6379	redis-cli ping
MongoDB 7	shopverse-mongo	27017	docker logs shopverse-mongo

Cassandra 4.1	shopverse-cassandra	9042	docker exec shopverse-cassandra cqlsh -e "DESCRIBE KEYSPACES"
Elasticsearch 8.13	shopverse-elasticsearch	9200	curl http://localhost:9200/_cluster/health
Kafka	shopverse-kafka	9092 / 29092	kafka-topics --list
Zookeeper	shopverse-zookeeper	2181	echo ruok nc localhost 2181
RabbitMQ 3.13	shopverse-rabbitmq	5672 / 15672 (UI)	rabbitmq-diagnostics ping
Neo4j 5	shopverse-neo4j	7474 / 7687	curl http://localhost:7474
Prometheus	shopverse-prometheus	9090	curl http://localhost:9090/-/healthy
Grafana	shopverse-grafana	3000	curl http://localhost:3000/api/health
Nginx	shopverse-nginx	80 / 443	curl http://localhost/health
Jenkins	shopverse-jenkins	8090	curl http://localhost:8090/login
Spring Boot App	shopverse-app	8080	curl http://localhost:8080/actuator/health
React UI	shopverse-ui	5173	curl http://localhost:5173

Verify all containers are running:

```
docker ps --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"
```

Section 3 - Starting the Application Layer

3.1 Spring Boot Backend

Requires infrastructure containers to be running first. App starts on port **8080**.

```
cd F:\Final\ShopVerse
./mvnw spring-boot:run -pl shopverse-web
```

Or with explicit dev profile:

```
./mvnw spring-boot:run -pl shopverse-web -Dspring-boot.run.profiles=dev
```

Health check after startup:

```
curl http://localhost:8080/actuator/health
```

Environment Variables (with defaults):

Variable	Default Value	Description
DB_HOST / DB_PORT	localhost / 5432	PostgreSQL host and port
DB_USER / DB_PASS	shopverse / shopverse	PostgreSQL credentials
REDIS_HOST / REDIS_PORT	localhost / 6379	Redis host and port
REDIS_PASS	(empty - no password)	Redis password
MONGO_URI	mongodb://localhost:27017/shopverse	MongoDB connection URI
CASSANDRA_HOST	localhost	Cassandra contact point
ES_URI	http://localhost:9200	Elasticsearch URI
KAFKA_BROKERS	localhost:9092	Kafka bootstrap servers
RABBITMQ_HOST	localhost	RabbitMQ host
RABBITMQ_PORT	5672	RabbitMQ AMQP port
RABBITMQ_USER	guest	RabbitMQ username
RABBITMQ_PASS	guest	RabbitMQ password
NEO4J_URI	bolt://localhost:7687	Neo4j Bolt URI
NEO4J_USER / NEO4J_PASS	neo4j / shopverse	Neo4j credentials
JWT_SECRET	shopverse-dev-secret-minimum-32-characters	JWT signing secret

3.2 React + Vite Frontend

The UI runs on port **5173** in dev mode.

```
cd F:\Final\ShopVerse\shopverse-ui  
npm install # first time only  
npm run dev # starts on http://localhost:5173
```

Build for production:

```
npm run build # output goes to shopverse-ui/dist/
```

Section 4 - Accessing & Querying Data

4.1 PostgreSQL

Connect via psql (inside Docker):

```
docker exec -it shopverse-postgres psql -U shopverse -d shopverse
```

Example Queries:

```
-- List all tables
\dt

-- View recent orders
SELECT id, customer_id, status, total, created_at
FROM orders ORDER BY created_at DESC LIMIT 10;

-- Count orders by status
SELECT status, COUNT(*) FROM orders GROUP BY status;

-- Products with low stock
SELECT id, name, sku, stock_count FROM products WHERE stock_count < 5;

-- Call stored procedure
CALL sp_complete_order(1);

-- View audit log (populated by trg_orders_audit trigger)
SELECT * FROM order_audit_log ORDER BY changed_at DESC LIMIT 5;
```

4.2 Redis

Connect via redis-cli (inside Docker):

```
docker exec -it shopverse-redis redis-cli
```

Example Commands:

```
# List all keys
KEYS *
```



```
# View a session token
GET session:<token>
```



```
# Inspect a shopping cart (Hash)
HGETALL cart:1
```



```
# View product leaderboard top 5 (Sorted Set)
ZREVRANGE leaderboard 0 4 WITHSCORES
```



```
# Rate limit counter
GET ratelimit:user:1
```



```
# Subscribe to inventory alerts (Pub/Sub)
SUBSCRIBE inventory:low
```



```
# Read from order event stream
XRANGE orders:events - + COUNT 10
```



```
# Check Redis memory usage
INFO memory
```

4.3 MongoDB

Connect via mongosh (inside Docker):

```
docker exec -it shopverse-mongo mongosh mongodb://localhost:27017/shopverse
```

Example Queries:

```
// List collections
show collections
```



```
// View recent reviews
db.reviews.find().sort({_id:-1}).limit(5).pretty()
```



```
// Reviews for a specific product
db.reviews.find({productId: '1'}).pretty()
```



```
// Average rating per product (aggregation pipeline)
db.reviews.aggregate([
  { $group: { _id: '$productId', avgRating: { $avg: '$rating' }, count: { $sum: 1 } } },
  { $sort: { avgRating: -1 } },
  { $limit: 10 }
])
```



```
// Full-text search on product documents
db.products.find({ $text: { $search: 'laptop' } })
```

4.4 Cassandra

Connect via cqlsh (inside Docker):

```
docker exec -it shopverse-cassandra cqlsh
```

Table Schema - order_activity:

```
USE shopverse;
DESCRIBE TABLE order_activity;

-- PRIMARY KEY structure:
-- Partition key : customer_id (all events for a customer on one node)
-- Clustering key : event_time DESC, event_id
-- Regular column : order_id, event_type, details

-- IMPORTANT: Always query by customer_id (the partition key).
-- Querying by order_id alone causes an ALLOW FILTERING error
-- because order_id is a regular column, NOT the partition key.
```

Correct Queries:

```
-- Describe keyspaces
DESCRIBE KEYSPACES;

-- All activity for a customer (correct - uses partition key)
SELECT * FROM order_activity WHERE customer_id = 1;

-- Latest 10 events for a customer (newest first)
SELECT * FROM order_activity WHERE customer_id = 1 LIMIT 10;

-- Events within a time range for a customer
SELECT * FROM order_activity
WHERE customer_id = 1
AND event_time >= '2024-01-01 00:00:00+0000'
AND event_time <= '2024-12-31 23:59:59+0000';

-- Set consistency level for writes
CONSISTENCY QUORUM;
```

How to query activities for a specific order (two-step approach):

Cassandra is partitioned by **customer_id**, NOT by **order_id**. Querying by **order_id** directly causes an ALLOW FILTERING error. The application uses a two-step approach:

- Step 1 - Look up the **customer_id** for the order from **PostgreSQL**
- Step 2 - Query Cassandra by **customer_id**, then filter by **order_id in memory** (Java stream filter)
- API endpoint: **GET /api/orders/{orderId}/activity** - controller calls `resolveCustomerId(orderId)` against PostgreSQL first, then fires the Cassandra query by `customer_id`

Replicating the two-step approach manually:

```
-- Step 1: get customer_id from PostgreSQL (run in psql)
SELECT customer_id FROM orders WHERE id = 152;
-- e.g. returns customer_id = 5

-- Step 2: query Cassandra using that customer_id (run in cqlsh)
SELECT * FROM shopverse.order_activity WHERE customer_id = 5;

-- Filter visually by order_id = 152 from the results
-- (the app does this in Java via stream().filter(a -> orderId.equals(a.getOrderId())))
```

Note: Do NOT run `SELECT * FROM order_activity WHERE order_id = 152` - this will always fail with an `ALLOW FILTERING` error because `order_id` is not the partition key.

4.5 Neo4j

Neo4j Browser: Open <http://localhost:7474> in your browser.

Bolt URL: bolt://localhost:7687 **User:** neo4j **Password:** shopverse

Graph Model (ShopVerse): Only **Product** nodes exist. Relationships are written by the Kafka consumer after each order is placed.

Node / Relationship	Written by	Meaning
Product	OrderKafkaConsumer	One node per product (upserted on every order)
FREQUENTLY_BOUGHT_TOGETHER	OrderKafkaConsumer	Pair of products in the same order — count increments each time
VIEWED_AFTER	ViewTrackingController	Product B viewed immediately after Product A in the same session

Inspect the Graph

```
// 1. See all nodes (visual graph in Browser)
MATCH (n) RETURN n LIMIT 50

// 2. Count nodes by label
MATCH (n) RETURN labels(n) AS label, count(n) AS count

// 3. Count relationships by type
MATCH ()-[r]->() RETURN type(r) AS type, count(r) AS count

// 4. See Product nodes with their relationships
MATCH (p:Product) OPTIONAL MATCH (p)-[r]-() RETURN p, r LIMIT 50
```

FREQUENTLY_BOUGHT_TOGETHER Edges

```
// All co-purchase pairs, ranked by how often bought together
MATCH (a:Product)-[r:FREQUENTLY_BOUGHT_TOGETHER]-(b:Product)
RETURN a.productId, a.name, b.productId, b.name, r.count
ORDER BY r.count DESC LIMIT 20

// Check a specific product node
MATCH (p:Product {productId: 1}) RETURN p

// 2-hop collaborative-filter recommendations for productId = 1
MATCH (p:Product {productId: 1})-[:FREQUENTLY_BOUGHT_TOGETHER*1..2]-(rec:Product)
WHERE rec.productId <> 1
RETURN DISTINCT rec.productId, rec.name, rec.avgRating
ORDER BY rec.avgRating DESC LIMIT 10
```

VIEWED_AFTER Edges

```
// Products viewed after another product
MATCH (a:Product)-[r:VIEWED_AFTER]->(b:Product)
RETURN a.productId, a.name, b.productId, b.name, r.count
ORDER BY r.count DESC LIMIT 20
```

Combined Recommendations (bought + viewed)

```
// Same query used by GET /api/products/{id}/recommendations
MATCH (p:Product {productId: 1})
```

```
# List all topics
kafka-topics --bootstrap-server localhost:29092 --list

# Output:
# shopverse.orders (order lifecycle events)
# shopverse.orders-retry-1000 (retry after 1s)
# shopverse.orders-retry-2000 (retry after 2s)
# shopverse.orders.DLT (dead-letter – failed after 3 retries)
# shopverse.inventory (stock reserve/release events)
# shopverse.inventory-retry-2000
# shopverse.inventory-retry-4000
# shopverse.inventory.DLT
# shopverse.analytics (user behaviour tracking – 6 partitions)
# shopverse.analytics-retry-500
# shopverse.analytics-retry-1000
# shopverse.analytics.DLT
# shopverse.products (product changes / ES sync)
# shopverse.notifications (email/notification events)
# __consumer_offsets (Kafka internal – ignore)
```

Describe a topic:

```
kafka-topics --bootstrap-server localhost:29092 --describe --topic shopverse.orders
kafka-topics --bootstrap-server localhost:29092 --describe --topic shopverse.inventory
kafka-topics --bootstrap-server localhost:29092 --describe --topic shopverse.analytics
kafka-topics --bootstrap-server localhost:29092 --describe --topic shopverse.products
```

Consuming messages (viewing actual message content):

```
# Read ALL messages from shopverse.orders from the beginning
kafka-console-consumer --bootstrap-server localhost:29092 \
--topic shopverse.orders --from-beginning

# Read messages and show the key as well
kafka-console-consumer --bootstrap-server localhost:29092 \
--topic shopverse.orders --from-beginning \
--property print.key=true --property key.separator=' | '

# Read only the latest 5 messages (most recent)
kafka-console-consumer --bootstrap-server localhost:29092 \
--topic shopverse.orders \
--offset latest --partition 0 --max-messages 5

# Read inventory events
kafka-console-consumer --bootstrap-server localhost:29092 \
--topic shopverse.inventory --from-beginning

# Read analytics events
kafka-console-consumer --bootstrap-server localhost:29092 \
--topic shopverse.analytics --from-beginning

# Read dead-letter messages (failed after 3 retries)
kafka-console-consumer --bootstrap-server localhost:29092 \
--topic shopverse.orders.DLT --from-beginning
```

Note on message format:

ShopVerse messages are serialised as **JSON** with a `__TypeId__` header that identifies the Java class (e.g. `OrderEvent$OrderPlaced`). The `kafka-console-consumer` will print the raw JSON payload. To also see headers (including `__TypeId__`):

```
kafka-console-consumer --bootstrap-server localhost:29092 \
--topic shopverse.orders --from-beginning \
--property print.headers=true \
--property print.key=true \
--property key.separator=' => '
```

Consume as a named consumer group:

```
# Join as the same group the app uses (will only get NEW messages)
kafka-console-consumer --bootstrap-server localhost:29092 \
--topic shopverse.orders --group shopverse-consumer

# Join as a NEW group (will get ALL messages from beginning)
kafka-console-consumer --bootstrap-server localhost:29092 \
--topic shopverse.orders --group my-debug-group --from-beginning
```

Check consumer lag:

```
# Check lag for the main order consumer group
kafka-consumer-groups --bootstrap-server localhost:29092 \
--group shopverse-consumer --describe

# Check lag for inventory consumer
kafka-consumer-groups --bootstrap-server localhost:29092 \
--group shopverse-inventory-consumer --describe

# List all consumer groups
kafka-consumer-groups --bootstrap-server localhost:29092 --list
```

Produce a test message:

```
# Start a producer for shopverse.orders
kafka-console-producer --bootstrap-server localhost:29092 \
--topic shopverse.orders
# Then type your JSON message and press Enter:
# {"orderId":1,"customerId":1,"type":"OrderPlaced"}
```

4.8 RabbitMQ Management UI

Management UI: Open <http://localhost:15672> in your browser.

Login with: Username **guest** Password **guest**

Navigating the Management UI:

- **Overview tab** - Shows message rates (publish/deliver/ack), total queued messages, and connected nodes.
- **Queues tab** - Lists all queues with message count, consumer count, and rate. Click any queue name to drill in.
- **Get Messages button** - Inside a queue detail page, click Get Message(s), set count, and click again to peek at payload without permanently consuming (Nack/requeue available).
- **Exchanges tab** - View exchange types (direct, topic, fanout) and their bindings to queues.
- **Bindings section** - Inside an exchange, shows which routing keys map to which queues.
- **Connections and Channels tabs** - Monitor active AMQP connections from the Spring Boot app.

AMQP connection string used by ShopVerse:

```
Host: localhost Port: 5672
Username: guest Password: guest
Virtual Host: /
```

CLI via rabbitmqadmin (inside container):

```
docker exec -it shopverse-rabbitmq bash

# List queues
rabbitmqadmin list queues name messages consumers

# List exchanges
rabbitmqadmin list exchanges

# Get a message from a queue
rabbitmqadmin get queue=<queue_name> ackmode=ack_requeue_true
```

4.9 Prometheus

Access Prometheus at <http://localhost:9090>

Spring Boot exposes metrics at <http://localhost:8080/actuator/prometheus>

Example PromQL Queries:

```
# HTTP request rate (per second, 5m window)
rate(http_server_requests_seconds_count[5m])

# p99 latency for all endpoints
histogram_quantile(0.99, rate(http_server_requests_seconds_bucket[5m]))

# JVM heap used
jvm_memory_used_bytes{area='heap'}

# JVM heap max
jvm_memory_max_bytes{area='heap'}

# HikariCP active connections
hikaricp_connections_active

# Kafka consumer lag
kafka_consumer_records_lag

# Order placement timer rate
rate(shopverse_order_placement_seconds_count[1m])
```

4.10 Grafana

Access Grafana at <http://localhost:3000>

Login with: Username **admin** Password **admin**

- **Dashboards** - Click the grid icon (left sidebar) then Dashboards then Browse to find the ShopVerse dashboard.
- **Order Throughput** - Requests per second for POST `/api/v1/orders`.
- **p99 Latency** - 99th-percentile response time across all API endpoints.
- **Error Rate** - Percentage of 5xx responses over time.
- **DB Connection Pool** - HikariCP active vs pending threads (alerts if pending > 5).
- **Kafka Consumer Lag** - Lag for shopverse-consumer group across all topics.
- **JVM Metrics** - Heap used, GC pause time, thread count.
- **Data Source** - Pre-configured to scrape from <http://shopverse-prometheus:9090>.

4.11 Spring Boot REST API

Base URL: <http://localhost:8080>

Register a new customer:

```
curl -X POST http://localhost:8080/api/auth/register \
-H "Content-Type: application/json" \
-d '{"username":"manish","email":"manish@example.com","password":"secret123"}'
```

Login and obtain JWT token:

```
curl -X POST http://localhost:8080/api/auth/login \
-H "Content-Type: application/json" \
-d '{"username":"manish","password":"secret123"}'
```

Copy the token from the response and use it in the Authorization header for all authenticated requests.

Search products (public):

```
curl 'http://localhost:8080/api/search?q=laptop'
```

Get all products (public):

```
curl http://localhost:8080/api/products
```

Place an order (authenticated):

```
curl -X POST http://localhost:8080/api/orders \
-H "Authorization: Bearer <YOUR_JWT_TOKEN>" \
-H "Content-Type: application/json" \
-d '{"items":[{"productId":1,"quantity":2}]}'
```

Get order activity (authenticated - two-step Cassandra lookup):

```
curl http://localhost:8080/api/orders/152/activity \
-H "Authorization: Bearer <YOUR_JWT_TOKEN>"
```

Internally: resolves customer_id from PostgreSQL, queries Cassandra by customer_id, then filters by order_id in memory.

Get product recommendations (authenticated):

```
curl http://localhost:8080/api/products/1/recommendations \
-H "Authorization: Bearer <YOUR_JWT_TOKEN>"
```

Section 5 - Quick Reference

Service	Port	URL / Connection	Default Credentials	Health Check
PostgreSQL	5432	jdbc:postgresql://localhost:5432/shopverse	shopverse / shopverse	pg_isready -U shopverse
Redis	6379	redis://localhost:6379	No password	redis-cli ping
MongoDB	27017	mongodb://localhost:27017/shopverse	No auth (dev)	mongosh --eval "db.runCommand({ping:1})"
Cassandra	9042	localhost:9042 keyspace: shopverse	No auth (dev)	cqlsh -e "DESCRIBE KEYSPACES"
Neo4j Browser	7474	http://localhost:7474	neo4j / shopverse	curl http://localhost:7474
Neo4j Bolt	7687	bolt://localhost:7687	neo4j / shopverse	-
Elasticsearch	9200	http://localhost:9200	No auth (dev)	curl localhost:9200/_cluster/health
Kafka	9092	localhost:9092 (external)	No auth (dev)	kafka-topics --list
RabbitMQ AMQP	5672	amqp://localhost:5672	guest / guest	rabbitmq-diagnostics ping
RabbitMQ UI	15672	http://localhost:15672	guest / guest	curl http://localhost:15672
Prometheus	9090	http://localhost:9090	No auth	curl localhost:9090/-/healthy
Grafana	3000	http://localhost:3000	admin / admin	curl localhost:3000/api/health
Jenkins	8090	http://localhost:8090	admin (wizard disabled)	curl http://localhost:8090/login
Nginx	80/443	http://localhost	-	curl http://localhost/health
Spring Boot	8080	http://localhost:8080	JWT via /api/auth/login	curl localhost:8080/actuator/health
React UI	5173	http://localhost:5173	-	curl http://localhost:5173